

Operational Models para Verificação de Brokers MQTT com ESBMC

Autor Placeholder

Departamento

Instituição

Cidade, País

email@exemplo.org

Resumo—A crescente adoção da Internet das Coisas (IoT) tem ampliado a superfície de ataque de sistemas críticos, muitos dos quais são escritos em C devido a requisitos de desempenho e recursos limitados. Brokers do protocolo MQTT são componentes centrais nesses ecossistemas e seu comprometimento pode afetar redes inteiras de dispositivos. Este trabalho propõe a extensão do verificador *Efficient SMT-Based Context-Bounded Model Checker* (ESBMC) por meio do desenvolvimento de *operational models* para APIs de rede e concorrência, com o objetivo de habilitar a análise formal de brokers MQTT escritos em C. O Eclipse Mosquitto é adotado como estudo de caso, e a avaliação se baseia na reprodução de *Common Vulnerabilities and Exposures* (CVEs) históricos do projeto, além de *benchmarks* científicos consolidados (Juliet Test Suite e SV-COMP).

Palavras-chave—verificação formal, model checking, ESBMC, MQTT, Mosquitto, IoT, segurança de software, vulnerabilidades em C, operational models.

I. INTRODUÇÃO

A. Contexto

A Internet das Coisas (IoT) deixou de ser uma promessa tecnológica para se consolidar como infraestrutura cotidiana, abrangendo desde aplicações de consumo até ambientes industriais e governamentais [1]. Estima-se que o número de dispositivos conectados ultrapasse a casa das dezenas de bilhões nos próximos anos, o que torna a comunicação entre esses dispositivos um problema de escala mundial.

Nesse contexto, o protocolo *Message Queuing Telemetry Transport* (MQTT) tornou-se o padrão de fato para troca de mensagens em ambientes de IoT, sobretudo pela sua leveza e por seu modelo de publicação/assinatura [2]. O elemento central de uma rede MQTT é o *broker*, responsável por receber, rotear e entregar mensagens aos clientes inscritos. Entre as implementações existentes, o *Eclipse Mosquitto* [3] é um dos brokers mais utilizados, mantido pela Eclipse Foundation e escrito em linguagem C.

A linguagem C, por sua vez, permanece dominante no desenvolvimento de software de sistemas e embarcado [4], oferecendo controle fino sobre memória e desempenho previsível, características indispensáveis em dispositivos com recursos restritos. Esse mesmo controle, contudo, transfere para o desenvolvedor a responsabilidade pelo gerenciamento de memória e pela ausência de comportamentos indefinidos, o

que historicamente tem se mostrado uma fonte recorrente de vulnerabilidades.

A verificação formal surge como uma técnica capaz de oferecer garantias matemáticas sobre o comportamento de um programa, complementando técnicas tradicionais como testes e análise estática [5]. Em particular, o *Efficient SMT-Based Context-Bounded Model Checker* (ESBMC) [6], [7] é uma ferramenta de *Bounded Model Checking* (BMC) que verifica programas C/C++ explorando seus caminhos até um limite finito de execução, traduzindo a obrigação de segurança para uma fórmula em lógica SMT a ser decidida por um *solver*.

B. Problema

Apesar dos avanços recentes em verificação formal, a aplicação direta do ESBMC a software de rede escrito em C, como brokers MQTT, esbarra em limitações práticas. A principal delas é a ausência de *operational models* adequados para APIs externas amplamente utilizadas nesse domínio, especialmente as APIs de sockets POSIX (`socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`), as APIs de multiplexação de entrada e saída (`select`, `poll`, `epoll`) e parte significativa das primitivas de concorrência da `pthread`.

Sem modelos abstratos para essas APIs, o ESBMC ou aborta a verificação por ausência de definição de símbolo, ou tenta seguir a implementação real do sistema operacional, o que rapidamente torna a exploração do espaço de estados inviável em termos de tempo e memória. O resultado prático é que uma classe inteira de software de infraestrutura, que inclui brokers, servidores e *daemons* de rede, permanece fora do alcance da verificação formal automatizada pelo ESBMC.

Esse panorama é particularmente crítico no contexto de IoT, em que vulnerabilidades em componentes de comunicação se propagam para milhões de dispositivos. O histórico recente do próprio Mosquitto registra *Common Vulnerabilities and Exposures* (CVEs) relacionados ao parser de pacotes MQTT, como o *integer overflow* no campo *Remaining Length* (CVE-2017-7651 [8]), *off-by-one* em rotinas de parsing de credenciais (CVE-2018-12551 [9]) e vazamentos de memória no fluxo de conexão (CVE-2017-7654 [10]). Tais vulnerabilidades poderiam, em princípio, ser detectadas por verificação formal, não fosse a barreira de modelagem mencionada.

O problema central abordado por este trabalho é, portanto:

Como estender o ESBMC, por meio de operational models para APIs de rede e concorrência, de modo a permitir a detecção automatizada de vulnerabilidades de memória em brokers MQTT escritos em C, com custo computacional viável e taxa de falsos positivos aceitável?

C. Motivação e Justificativa

A justificativa para investir em verificação formal de brokers MQTT articula-se em três eixos complementares.

Eixo de impacto. Brokers MQTT são pontos únicos de falha em arquiteturas IoT típicas. O comprometimento de um broker permite, em geral, interceptar, alterar ou injetar mensagens em toda a rede de dispositivos conectados. Detectar vulnerabilidades antes do *deploy*, em vez de reagir a explorações após a divulgação pública, reduz substancialmente o custo e o risco associados à operação desses sistemas.

Eixo científico. *Operational models* são um recurso bem estabelecido em verificação formal para abstrair comportamentos externos a fim de tornar a análise tratável [11]. Apesar disso, o catálogo de modelos efetivamente disponível no ESBMC para APIs de rede e concorrência ainda é incompleto, especialmente para operações assíncronas e multiplexação. Preencher essa lacuna não beneficia apenas a análise de brokers MQTT, mas habilita o ESBMC a tratar toda uma classe de software de rede com a mesma metodologia.

Eixo prático e de reprodutibilidade. Diferentemente de modificar o núcleo em C++ do ESBMC, o desenvolvimento de *operational models* ocorre em arquivos C dentro do diretório `src/c2goto/library/` do projeto, com curva de aprendizado e risco de regressão menores. A contribuição resultante pode ser submetida ao repositório oficial do ESBMC como *patch*, beneficiando a comunidade. Além disso, ao adotar o Eclipse Mosquitto, que é *open source* e amplamente documentado, garante-se a reprodutibilidade do estudo experimental por outros pesquisadores.

A combinação desses três eixos torna o problema escolhido relevante tanto do ponto de vista da segurança aplicada quanto do ponto de vista da pesquisa em verificação formal.

D. Objetivos

1) *Objetivo Geral:* Desenvolver e avaliar uma extensão do ESBMC, baseada em *operational models* para APIs de rede e concorrência, com o intuito de habilitar a detecção automatizada de vulnerabilidades de memória em brokers MQTT escritos em linguagem C, utilizando o Eclipse Mosquitto como estudo de caso.

2) *Objetivos Específicos:*

- 1) Identificar, a partir do código-fonte do Mosquitto e de execuções preliminares do ESBMC, o conjunto mínimo de APIs externas cuja ausência de modelagem impede ou inviabiliza a verificação do broker.
- 2) Projetar e implementar *operational models* para essas APIs no diretório de biblioteca do ESBMC, observando o equilíbrio entre abstração suficiente para garantir

verificabilidade e precisão suficiente para evitar falsos positivos.

- 3) Construir *harnesses* de verificação para subsistemas críticos do Mosquitto, em especial o parser do cabeçalho MQTT, a leitura do campo *Remaining Length* e o tratamento inicial do pacote `CONNECT`.
- 4) Estabelecer uma linha de base científica utilizando *benchmarks* consolidados, especificamente um subconjunto da *Juliet Test Suite* (CWE-121, CWE-122, CWE-416 e CWE-476) e do repositório do SV-COMP nas categorias *ReachSafety* e *MemSafety*.
- 5) Reproduzir, com o ESBMC modificado, CVEs históricos selecionados do Mosquitto, mensurando a taxa de detecção em comparação com o ESBMC *vanilla*.
- 6) Avaliar quantitativamente o impacto da extensão proposta em termos de tempo de verificação, consumo de memória, taxa de *timeouts*, taxa de detecção verdadeira (*true positives*) e taxa de falsos positivos.
- 7) Documentar diretrizes práticas para o desenvolvimento e a manutenção de novos *operational models* no ESBMC, de modo a facilitar contribuições futuras da comunidade.

REFERÊNCIAS

- [1] Statista Research Department, “Number of Internet of Things (IoT) Connected Devices Worldwide,” <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/>, 2024, accessed: 2026-05-11.
- [2] OASIS, “MQTT Version 5.0 – OASIS Standard,” <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019, accessed: 2026-05-11.
- [3] Eclipse Foundation, “Eclipse Mosquitto: An Open Source MQTT Broker,” <https://mosquitto.org/>, accessed: 2026-05-11.
- [4] TIOBE Software BV, “TIOBE Programming Community Index,” <https://www.tiobe.com/tiobe-index/>, accessed: 2026-05-11.
- [5] E. M. Clarke, T. A. Henzinger, H. Veith, and R. Bloem, Eds., *Handbook of Model Checking*. Springer, 2018.
- [6] L. Cordeiro, B. Fischer, and J. Marques-Silva, “SMT-Based Bounded Model Checking for Embedded ANSI-C Software,” *IEEE Transactions on Software Engineering*, vol. 38, no. 4, pp. 957–974, 2012.
- [7] M. R. Gadelha, F. R. Monteiro, J. Morse, L. C. Cordeiro, B. Fischer, and D. A. Nicole, “ESBMC 5.0: An industrial-strength C model checker,” *International Journal on Software Tools for Technology Transfer*, vol. 21, no. 3, pp. 323–341, 2017.
- [8] MITRE, “CVE-2017-7651: Integer Overflow in Eclipse Mosquitto MQTT Broker,” <https://nvd.nist.gov/vuln/detail/CVE-2017-7651>, 2017, accessed: 2026-05-11.
- [9] —, “CVE-2018-12551: Authentication Bypass in Eclipse Mosquitto,” <https://nvd.nist.gov/vuln/detail/CVE-2018-12551>, 2018, accessed: 2026-05-11.
- [10] —, “CVE-2017-7654: Memory Leak in Eclipse Mosquitto,” <https://nvd.nist.gov/vuln/detail/CVE-2017-7654>, 2017, accessed: 2026-05-11.
- [11] C. L. Heitmeyer, R. D. Jeffords, and B. G. Labaw, “Automated Consistency Checking of Requirements Specifications,” in *ACM Transactions on Software Engineering and Methodology*, vol. 5, no. 3, 1996, pp. 231–261.